

Secure IoT Communication

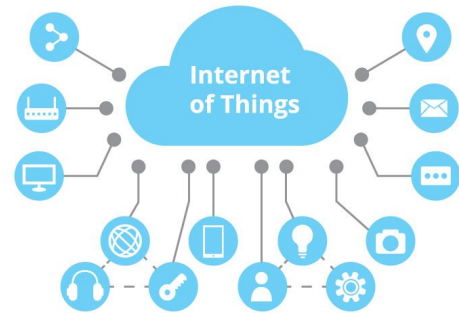
By: Michael Woldai &
Carter Trudeau



How to transmit sensor readings without fear of manipulation?

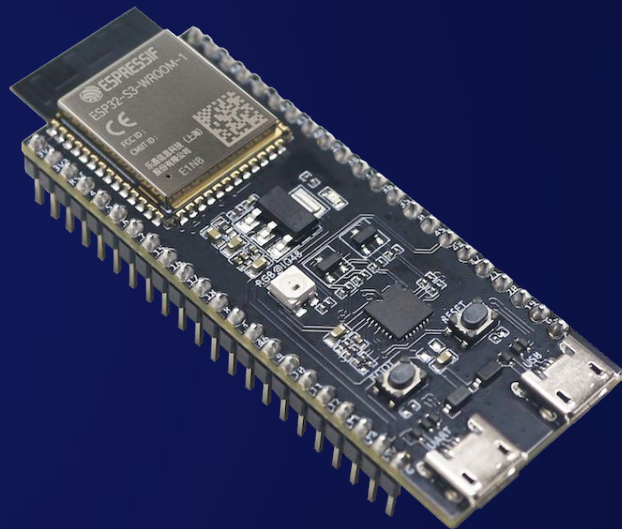
- IoT devices transmit sensitive sensor data
- Unencrypted wireless links expose users to eavesdropping and tampering

Goal: Add a secure communication layer to an embedded system that can then be used to help future projects



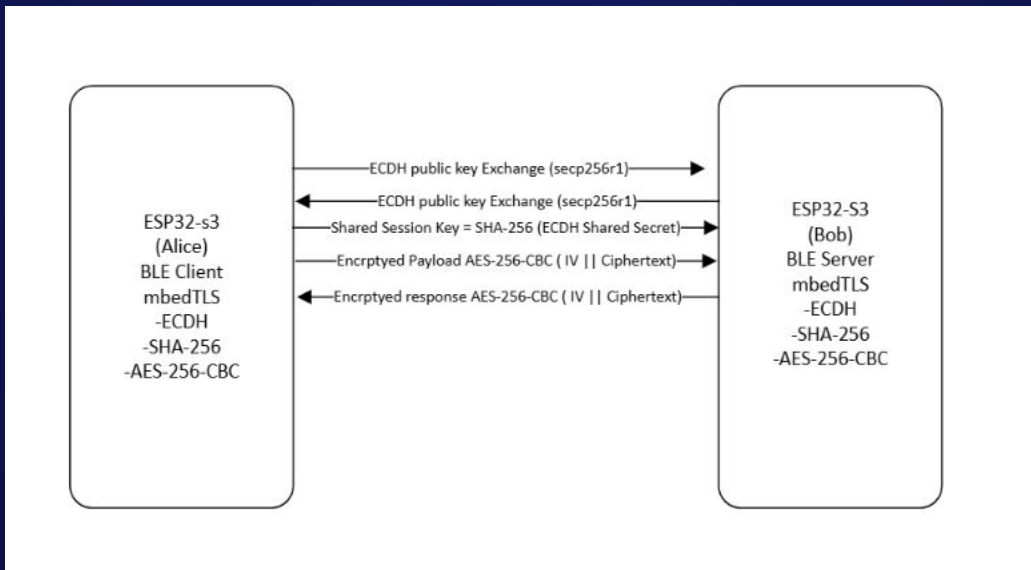
How We Plan To Implement

- Asymmetric Key Exchange
 - ECDH on curve secp256r1 to create a shared secret
- Symmetric Encryption
 - AES-256
- Key Derivation
 - SHA-256 to convert ECDH to a consistent key
- Implemented over Bluetooth Low Energy (BLE) on 2 ESP32-S3 dev boards



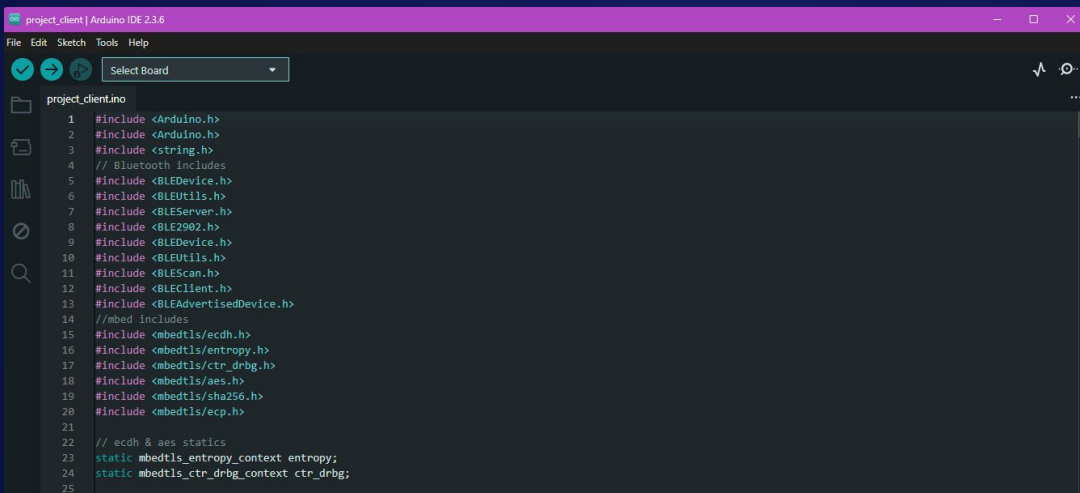
System Design Overview

- ESP32-S3 devices communicate over BLE
- Exchange EC public keys
- Perform ECDH to derive AES-256 key
- Secure encrypted messages can now be sent between client and server



Implementation

- Implemented in C++ using Arduino IDE
 - Using ESP32 BLE libraries and mbedTLS libraries
- Random IV generated for each ciphertext
- Communication with the device done over serial so we can view the messages
- Both client and server print the plaintext and ciphertext to serial for verification
 - Only ciphertext is sent over bluetooth



```
1 #include <Arduino.h>
2 #include <Arduino.h>
3 #include <string.h>
4 // Bluetooth includes
5 #include <BLEDevice.h>
6 #include <BLEUtils.h>
7 #include <BLEServer.h>
8 #include <BLE2902.h>
9 #include <BLEDevice.h>
10 #include <BLEUtils.h>
11 #include <BLEScan.h>
12 #include <BLEClient.h>
13 #include <BLEAdvertisedDevice.h>
14 //mbed includes
15 #include <mbedtls/ecdh.h>
16 #include <mbedtls/entropy.h>
17 #include <mbedtls/ctr_drbg.h>
18 #include <mbedtls/aes.h>
19 #include <mbedtls/sha256.h>
20 #include <mbedtls/ecp.h>
21
22 // ecdh & aes statics
23 static mbedtls_entropy_context entropy;
24 static mbedtls_ctr_drbg_context ctr_drbg;
25
```

Sending Messages

Output Serial Monitor X

Message (Enter to send message to 'SparkFun ESP32-S3 Thing Plus' on 'COM4')

New Line 115200 baud

Starting BLE Chat Client...
Found target server, stopping scan
Connecting to server at: 98:a3:16:f5:36:9d
Connected to server
Got server public key (65 bytes)
Shared AES key derived on client.
Client public key sent to server.
Ready to chat!
Received ciphertext: 79FF3C38A656165260F200D3EB2D63A88B2EA32B1A795D5C21AB66DCED2FBD51C280200B01AFE997C45F82CF298551333F4D01D29D4ACF2E2021A251E38E913A
Decrypted text: The quick brown fox jumps over the lazy dog
Msg to send: THE LAZY YELLOW DOG WAS CAUGHT BY THE SLOW RED FOX AS HE LAY SLEEPING IN THE SUN
Sending ciphertext: 4745D70E8D88973DF254B08AA877F9280E2CC5FB87D4447111B7F73D0400351B0D7AA9CC73BD35719CBCB4F6964CEC6F0B8A2277F635821DBAF4089300F1FF47E8D22CC4D0FB877B

Ln 1, Col 1 SparkFun ESP32-S3 Thing Plus on COM4 2

Output Serial Monitor X

Message (Enter to send message to 'SparkFun ESP32-S3 Thing Plus' on 'COM6')

New Line 115200 baud

Starting BLE Chat Server...
BLE Chat Server is advertising
Client connected
Received client ECDH public key
Shared AES key derived. Handshake complete.
Msg to send: The quick brown fox jumps over the lazy dog
Sending ciphertext: 79FF3C38A656165260F200D3EB2D63A88B2EA32B1A795D5C21AB66DCED2FBD51C280200B01AFE997C45F82CF298551333F4D01D29D4ACF2E2021A251E38E913A

Received ciphertext: 4745D70E8D88973DF254B08AA877F9280E2CC5FB87D4447111B7F73D0400351B0D7AA9CC73BD35719CBCB4F6964CEC6F0B8A2277F635821DBAF4089300F1FF47E8D22CC4D0FB877B
Decrypted text: THE LAZY YELLOW DOG WAS CAUGHT BY THE SLOW RED FOX AS HE LAY SLEEPING IN THE SUN

Lessons Learned

- Secure computation requires careful attention to data formats and key sizes
- BLE introduces unique constraints compared to WiFi/TLS
- Default bluetooth libraries actually already include encrypted communications but BLE does not



Team Contributions

Michael Woldai

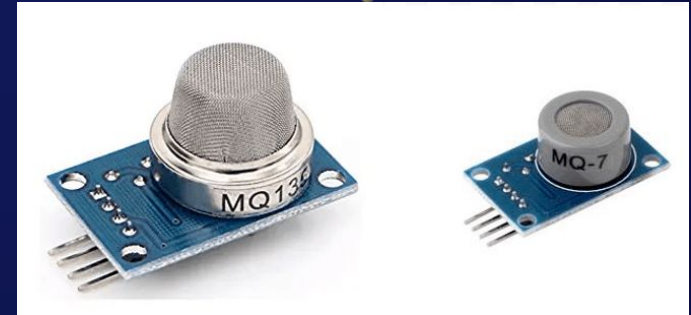
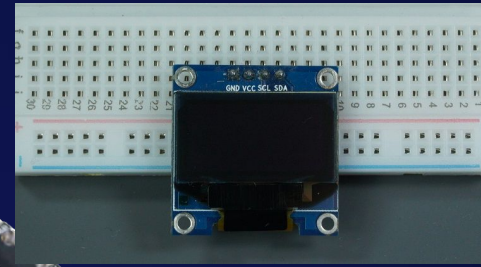
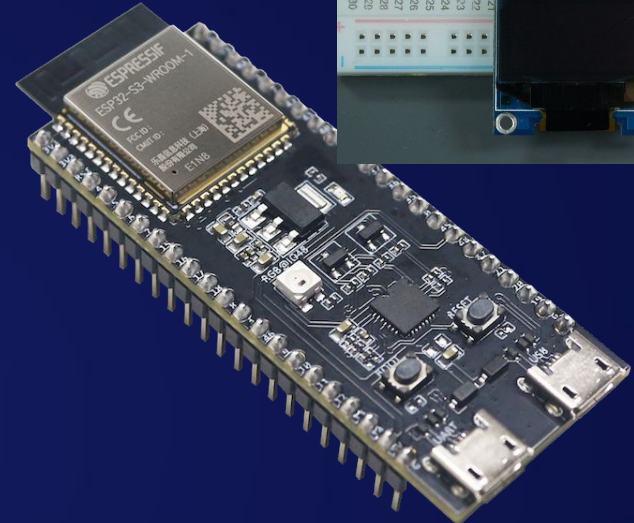
- Air-quality monitor integration, BLE service structure
- Encryption pipeline development
- Testing and debugging of AES key mismatch issue

Carter Trudeau

- ECDH implementation and key derivation
- BLE client-server protocol design
- mbedTLS integration and serial diagnostics

Future Work

- Add certificate-based identity verification
- Expand to multi-node IoT networks or mesh BLE
- Implement session re-establishment so the devices can power off independently and resume encrypted communication
- Integrate sensors and independent screens so devices can record and transmit data to one and another
- Migrate to a different way of compiling and flashing the device to improve file readability
- Use RTOS to have it automatically transmit data when sensors detect a problem



Questions?